mid term Report

ruthvik

20-6-2026

Contents

1 Progress **3**

2 Things I Learned **3**

2.1 Time Complexity Analysis . . . 3

2.2 Selection Sort . . . 4

2.3 Bubble Sort . . . 4

2.4 Insertion Sort . . . 5

2.5 Insertion Sort Implementation . . . 5

2.6 Merge Sort . . . 5

2.7 Merge Sort Implementation . . . 6

2.8 Quick Sort . . . 6

2.9 Binary Search . . . 7

2.10 Structures . . . 8

2.11 Classes and Constructors (Basic Understanding) . . . 8

2.12 Two Pointer Technique . . . 8

2.13 Two Pointer Technique . . . 8

2.14 Sliding Window Technique . . . 9

2.15 Sliding Window Technique . . . 9

2.16 Sets and Unordered Sets (Basic Understanding) . . . 9

2.17 Stacks and Queues . . . 10

2.18 STL for Stack and Queue 10

2.19 Deque, Circular Queue, and Simple Queue 10

3 Way I Did **10**

1 Progress

In this phase of my learning, I focused on building a strong base in core data structure concepts. At the beginning, most topics looked difficult and disconnected. But after studying them one by one and practicing the basic ideas, the topics started to make more sense.

I first worked on understanding how to think about time complexity, because that helps in comparing solutions and choosing better ones. After that, I learned the basic sorting algorithms like selection sort, bubble sort, and insertion sort. These helped me understand how arrays can be organized step by step.

Then I moved to slightly more advanced topics such as merge sort and quick sort. These made me see how recursion can be used to solve bigger problems by breaking them into smaller parts.

Later, I studied structures, classes, and constructors to understand how data can be grouped in a neat way. I also learned common problem-solving techniques such as two pointers and sliding window, which are very useful for array and string problems.

Finally, I studied sets and unordered sets, and then stacks, queues, and their STL versions. These topics helped me understand how to manage data in different orders depending on the problem.

2 Things I Learned

2.1 Time Complexity Analysis

Time complexity tells us how much time an algorithm may take as the input size grows. It is not about the exact clock time. It is about how the work increases when the input becomes larger.

A simple idea is this: if one method checks every element again and again, it will usually take more time than a method that reduces the work step by step. Big-O notation is used to describe this growth.

- **O(1)** means constant time, which means the work does not depend much on input size.
- **O(n)** means the time grows linearly with the input.
- **O(n^2)** means the time grows very fast, usually because of nested loops.

This concept is important because it helps us choose efficient solutions instead of slow ones.

2.2 Selection Sort

Selection sort is one of the simplest sorting algorithms. It works by finding the smallest element from the unsorted part and placing it at the beginning.

Imagine a row of numbers. You scan the whole row, pick the smallest one, and swap it into the first position. Then you repeat the same process for the remaining part.

It is easy to understand, but it is not very fast for large data. Its time complexity is usually **O(n^2)**.

```
int posMax(int* a, int n){
    int maxInd = 0;
    for(int i=0;i<n;i++) if(a[maxInd]<a[i]) maxInd=i;
    return maxInd;
}

void selsort(int* a, int n ){
    for (int i=n; i>1;i--){
        int temp = a[i-1];
        a[i-1] = a[posMax(a,i)];
        a[posMax(a,i)] = temp;
    }
}

void rselsort(int* a, int n){
    if(n==1) return;
    else{
        int temp = a[n-1];
        a[n-1] = a[posMax(a,n)];
        a[posMax(a,n)] = temp;
        rselsort(a,n-1);
    }
}
```

2.3 Bubble Sort

Bubble sort compares neighboring elements and swaps them if they are in the wrong order. After one full pass, the largest element moves to the end, like a bubble rising to the top.

This algorithm is simple and good for learning the basic idea of swapping and repeated passes. But like selection sort, it is slow for large inputs, with time complexity **O(n^2)**.

```
void swap(int &a, int &b){//& means we are passing the variables by
   reference, so any changes made to a and b inside the function will
   affect the original variables passed to the function.
    int temp = a;
    a = b;
```

```
    b = temp;
}

void bubsort(int* a,int n){
    //no of watys of selecting 2 are n-1 ways so
    for(int j=0;j<n-1;j++){//we have to do n-1 times
        for (int i=1; i<n-j; i++){//after j times the last j elements
           are in correct position so we can ignore them
        if (a[i-1]>a[i]){
            swap(a[i-1], a[i]);
        }
        else continue;
    }}
}
```

2.4 Insertion Sort

Insertion sort builds the sorted part gradually. It takes one element at a time and inserts it into the correct position among the already sorted elements.

This is similar to arranging playing cards in your hand. You pick up one new card and place it in the right spot.

It works well for small or nearly sorted arrays and is also easy to understand.

2.5 Insertion Sort Implementation

```
void inssort(int* a, int n){
    for (int i=0;i<n;i++){
        int ins =a[i];
        int j = i-1;
        while(j>=0&&ins<a[j]){
            a[j+1]=a[j];
            j--;//j can become negative
        }
    a[j+1]=ins;
    }
}
```

2.6 Merge Sort

Merge sort uses the divide-and-conquer approach. It splits the array into two halves, sorts each half separately, and then merges the two sorted halves.

This method is much more efficient than the simple quadratic sorting methods. Its time complexity is **O(n log n)**.

The main idea here is important: break a big problem into smaller pieces, solve them, and combine the answers.

2.7 Merge Sort Implementation

```
void mergeSortedParts(vector<int>& a, int l, int mid, int r) {
    vector<int> temp;
    int i = l, j = mid + 1;

    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else temp.push_back(a[j++]);
    }
    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, l, mid);
    mergeSort(a, mid + 1, r);
    mergeSortedParts(a, l, mid, r);
}
```

2.8 Quick Sort

Quick sort also uses divide and conquer. It selects a pivot element and rearranges the array so that smaller elements come before the pivot and larger elements come after it.

Then it repeats the same process on both sides.

Quick sort is often very fast in practice, and its average time complexity is **O(n log n)**. It is a key algorithm because it teaches partitioning and recursive thinking.

```
int partitionLomuto(vector<int>& a, int l, int r) {
    int pivot = a[r];
    int i = l - 1;
    for (int j = l; j < r; j++) {
        if (a[j] <= pivot) {
```

```
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[r]);
    return i + 1;
}

void quickSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int p = partitionLomuto(a, l, r);
    quickSort(a, l, p - 1);
    quickSort(a, p + 1, r);
}
```

2.9 Binary Search

Binary Search is an efficient searching algorithm that works only on sorted arrays.

Instead of checking every element one by one, Binary Search repeatedly divides the search space into two halves. It compares the target value with the middle element.

- If the target is equal to the middle element, the search is complete.
- If the target is smaller, we search in the left half.
- If the target is larger, we search in the right half.

Because the search space becomes half after every step, Binary Search is much faster than Linear Search for large datasets.

The time complexity of Binary Search is **O(log n)**.

A key requirement is that the data must be sorted before Binary Search can be applied.

```
int binarySearchIterative(const vector<int>& a, int key) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;   // safer than (l + r) / 2
        if (a[mid] == key) return mid;
        if (a[mid] < key) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}
```

2.10 Structures

A structure is a way to group related data together. Instead of storing many separate variables, we can keep them inside one unit.

For example, if we want to represent a student, we may store name, roll number, and marks together in a structure.

This makes code cleaner and easier to manage.

2.11 Classes and Constructors (Basic Understanding)

A class is similar to a structure, but it also helps us combine data and functions together. It is one of the main ideas of object-oriented programming.

A constructor is a special function that runs automatically when an object is created. Its job is to initialize the object with proper values.

This concept is useful because it helps make programs more organized, reusable, and easier to understand.

2.12 Two Pointer Technique

The two pointer technique uses two indexes to solve a problem more efficiently. These pointers may move in the same direction or from opposite ends, depending on the problem.

For example, one pointer may start at the left and the other at the right. By moving them carefully, we can reduce extra work.

This technique is very common in array and string problems, and it often helps reduce time complexity.

2.13 Two Pointer Technique

Finding whether a sorted array contains two numbers whose sum is equal to a target.

```
bool hasPair(vector<int>& arr, int target) {
    int left = 0;
    int right = arr.size() - 1;

    while(left < right) {
        int sum = arr[left] + arr[right];

        if(sum == target)
            return true;

        if(sum < target)
```

```
            left++;
        else
            right--;
    }

    return false;
}
```

2.14 Sliding Window Technique

Sliding window is used when we want to process a continuous part of an array or string. Instead of checking every possible range from scratch, we keep a window and move it step by step.

This saves time because we update the current answer instead of recalculating everything again.

It is useful in problems involving subarrays, substrings, sums, and counts.

2.15 Sliding Window Technique

Finding the maximum sum subarray of size k.

```
int maxSumSubarray(vector<int>& arr, int k) {
    int windowSum = 0;

    for(int i = 0; i < k; i++)
        windowSum += arr[i];

    int maxSum = windowSum;

    for(int i = k; i < arr.size(); i++) {
        windowSum += arr[i];
        windowSum -= arr[i - k];

        maxSum = max(maxSum, windowSum);
    }

    return maxSum;
}
```

2.16 Sets and Unordered Sets (Basic Understanding)

A set stores unique elements only. This means duplicates are automatically avoided.

A set keeps elements in sorted order, while an unordered set does not keep a fixed order but usually gives faster access.

These data structures are useful when we want to check whether something already exists, remove duplicates, or store distinct values efficiently.

2.17 Stacks and Queues

A stack follows the **LIFO** rule, which means *Last In, First Out*. The last element added is the first one removed.

A queue follows the **FIFO** rule, which means *First In, First Out*. The first element added is the first one removed.

These structures are very important because many real problems need controlled insertion and removal of data.

2.18 STL for Stack and Queue

STL gives ready-made implementations of stack and queue in C++. This saves time and reduces coding mistakes.

Instead of building everything from scratch, we can use the built-in library versions when the problem does not require a custom implementation.

2.19 Deque, Circular Queue, and Simple Queue

A deque is a double-ended queue. It allows insertion and deletion from both the front and the back.

A simple queue allows insertion at one end and deletion from the other end.

A circular queue is a more efficient queue design where the last position connects back to the first position. This helps in using space properly and avoiding waste.

These topics taught me how different data handling rules can be used depending on the type of problem.

3 Way I Did

1. First, I read the concept in a basic way so I could understand the main idea.
2. Then, I wrote small examples by hand to see how the algorithm or data structure behaves.
3. After that, I practiced coding did few easy and very easy problems to understand the concept better
4. I compared similar topics, such as selection sort, bubble sort, and insertion sort, to see how they are different.

5. I also revisited topics like time complexity again and again.

This method helped me move from only reading theory to actually understanding how the concepts work in practice.